

Data Structures & Algorithms — Revision Notes

Data structures, complexity analysis and the core algorithms every interview expects.

1. Complexity at a glance

Big-O describes how running time or memory grows as input size n grows. Constants and lower-order terms are dropped.

Structure	Access	Search	Insert	Delete
Array (static)	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Dynamic array	$O(1)$	$O(n)$	$O(1)$ amortised	$O(n)$
Singly linked list	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash table	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary heap	—	$O(n)$	$O(\log n)$	$O(\log n)$

2. Arrays & two pointers

- Prefix sums turn repeated range-sum queries into $O(1)$ work after an $O(n)$ pre-pass.
- Two pointers works on sorted data: move the pointer that can still improve the result.
- Sliding window maintains a running invariant over a contiguous range — expand right, shrink left.
- In-place partitioning (Dutch national flag) sorts three-valued arrays in one pass.

```
for (let l = 0, r = n - 1; l < r; ) {  
  const sum = a[l] + a[r];  
  if (sum === target) return [l, r];  
  sum < target ? l++ : r--;  
}
```

3. Linked lists

- Sentinel/dummy head nodes remove almost every null-edge special case.
- Fast and slow pointers detect cycles (Floyd) and find the middle node.
- Reverse iteratively with three pointers: prev, curr, next.
- Merge two sorted lists by advancing the list with the smaller head value.

4. Stacks, queues & heaps

- Monotonic stack: next greater element, largest rectangle in histogram, stock span.
- Deque gives amortised $O(1)$ sliding-window maximum.
- Binary heap supports push/pop in $O(\log n)$ and peek in $O(1)$; heapify is $O(n)$.
- Top-k problems: keep a min-heap of size k , not a full sort.

5. Hash tables

A hash table maps keys to buckets through a hash function; collisions are resolved by chaining or open addressing.

- Load factor above ~ 0.7 triggers rehashing — keep the table sized to $\sim 2x$ the entry count.
- Counting problems are almost always hash-map problems: frequencies, first duplicate, anagrams.
- Prefix-sum + hash map solves subarray-sum-equals-k in $O(n)$.

6. Trees

- Traversals — preorder (copy), inorder (sorted output for BST), postorder (delete/free), level order (BFS).
- BST operations are $O(h)$; h is $O(\log n)$ balanced, $O(n)$ degraded.
- AVL and red-black trees keep $h = O(\log n)$ with rotations.
- Trie: prefix search, autocomplete, word dictionaries in $O(\text{key length})$.

7. Graphs

- Represent as adjacency list (sparse) or adjacency matrix (dense).
- BFS finds shortest paths on unweighted graphs; DFS suits connectivity and cycle detection.
- Dijkstra: non-negative weights, $O((V + E) \log V)$ with a binary heap.
- Bellman-Ford handles negative weights in $O(V \cdot E)$ and detects negative cycles.
- Topological sort via Kahn's algorithm detects cycles in directed graphs.
- Union-Find with path compression near $O(1)$ per operation for connectivity.

8. Sorting & searching

Algorithm	Average	Worst	Stable	Notes
Bubble / Insertion	$O(n^2)$	$O(n^2)$	Yes	Insertion sort wins on nearly sorted data
Merge sort	$O(n \log n)$	$O(n \log n)$	Yes	Linked lists, external sort
Quick sort	$O(n \log n)$	$O(n^2)$	No	In-place; randomise the pivot
Heap sort	$O(n \log n)$	$O(n \log n)$	No	$O(1)$ extra space
Counting / Radix	$O(n + k)$	$O(n + k)$	Yes	Small integer key ranges

9. Dynamic programming checklist

- Define the state precisely, then write the recurrence before coding.
- Identify the base case and the iteration order that fills dependencies first.
- Classic patterns: 0/1 knapsack, LCS, edit distance, coin change, LIS, grid paths.
- Optimise space when the recurrence only needs the previous row.